# High-Performance Conjugate Gradient on GPUs and CPUs: Optimisation and Scaling Laws

LOC TUNG SU, University of New Mexico, USA
ELIUD GARCÍA REZA, University of New Mexico, USA
STEVEN UPSTON, University of New Mexico, USA

This report covers our results and analysis of running and optimizing High-Performance Conjugate Gradient (HPCG) with a variety of configurations. The report will include a Strong Scaling study to analyse the effects of adding more processing power on run time, a Weak Scaling study to analyse the effects of adding more processing power on scaling up the problem size, and various studies into the effects of varying problem sizes, run times, and number of nodes for CPUs and GPUs.

**ACM Reference Format:**
Loc Tung Su, Eliud García Reza, and Steven Upston. 2026. High-Performance Conjugate Gradient on GPUs and CPUs: Optimisation and Scaling Laws. 1, 1 (June 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## Introduction

High-Performance Conjugate Gradient (HPCG) is a benchmarking software that measures the performance of a compute cluster by solving a sparse linear system of equations using a conjugate gradient algorithm. Optimising and analysing the performance of HPCG with GPUs in comparison to CPUs is the main goal of this project. Our analysis includes studies for Amdahl's Law, Gustafson's Law, and various studies comparing CPU and GPU performance.

### 0.1 Amdahl's Law (Strong Scaling)

Amdahl's Law describes how a fixed-size problem eventually reaches a point of diminishing returns due to the serial fraction of the computation. In the context of our project, this means running HPCG with varying CPU and GPU counts to see the effect it has on the performance. Our goal with this study is to discover what part of HPCG is strictly serialized and if we can get HPCG to run as close to the ideal linear speed-up as possible.

### 0.2 Gustafson's Law (Weak Scaling)

Gustafson's Law describes ideal speedup as a function of process count and problem size. In the context of our project, this means increasing the problem size of HPCG while keeping the amount of work done by each process constant. Our goal with this study is to confirm that when scaling up the problem size of HPCG that the speedup can remain at least semi-linear if the number of processes is scaled up to match the problem size.

Authors' Contact Information: Loc Tung Su, University of New Mexico, Albuquerque, NM, USA; Eliud García Reza, University of New Mexico, Albuquerque, NM, USA; Steven Upston, University of New Mexico, Albuquerque, NM, USA.


ACM XXXX-XXXX/2026/6-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

### 0.3 Various Studies

Besides a strong and weak scaling study, we were tasked with comparing GPU and CPU HPCG performance using varying problem sizes. Our goal with this is to see how CPUs and GPUs handle different problem sizes, and if anything can be gleaned about how HPCG works and what it truly measures. In a similar vein, we also ran HPCG with varying numbers of nodes, intending to see how HPCG works when a bottleneck like internode communication is added into the equation. Finally we also conducted a study into the effects that varying HPCG runtime might have on performance, and if there is any pattern or scaling that can be determined from the data.

## Methods

In this section, we demonstrate the methodology we used to analyze performance using the **High Performance Conjugate Gradient** (HPCG) on both GPUs and CPUs. We explore the performance on Easley. For CPUs, we used the **general** and **debug** partition, and for GPUs, we used both **H100** and **L40s** partitions.

### 0.4 General setup

With the provided HPCG repository, in order to start collecting the data we're looking for, we need to modify the SLURM sbatch script, load apptainer with `module load apptainer`, download the NVIDIA benchmarks container (24.03), navigate the container and copy hpcg.sh from the container to host system. Additionally, we created 4 new files, `PARAM_PARAM.csv`, `HPCG_TEMPLATE.dat`, **full_job_output** folder, `RESULTS_FULL.csv` and `RESULTS_SML.csv`. `HPCG_TEMPLATE.dat` is HPCG benchmark configuration template file that takes on 4 parameters: NX, NY, NZ which represents the problem dimensions, and RT (run time). `PARAM_PARAMS.csv` file specifies the parameters for each job submission that ultimately will be parsed in `HPCG_TEMPLATE.dat`. Each submission can perform multiple runs, and each row represents one benchmark run where each column describes NX → NY → NZ → RT. Both `RESULTS_FULL.csv` and `RESULTS_SML.csv` store our output results. Lastly, **full_job_output** folder will store all the SLURM job outputs.
We have 2 different slurm scripts, one for CPUs and one for GPUs, and both scripts require a directory changes from Ryan's directory to our local directory for CONT, MOUNT, RESULTS_FULL, RESULTS_SML, etc.
For GPUs, we need to load these module:

- gcc/14.2.0-cuda-jeua
- apptainer

And for CPUs, we need to:

- load openmpi/4.1.7-3ilj
- load hpcg/3.1-hchi
- changes our run commands to "mpirun -np $((SLURM_NNODES * SLURM_NTASKS_PER_NODE)) $HPCG_BIN/xhpcg » $OUTPUT".

However, with our CPUs case, the run command above despite it does not give us any error in our output, it also fail to output the GFLOPS result in our output file. We have to manually locate the correspond "**tmp** file → HPCG-Benchmark.txt and look for the GFLOP results in the last few lines of the text file.

### 0.5 Amdahl scaling

For our strong scaling experiment, we designed an experiment to approximate Amdahl's law on only Easley cluster. We ran the experiment using one fixed problem dimension (144, 144, 144) with 1 fix run time (300). The reason why we went with that problem size is because it is the largest problem size we can test without causing OOM issue. We perform this experiment on both CPUs and GPUs. On GPUs, we ran it all on 1 node with 1/2/3 GPUs on the L40s partition, and only with 1/2 GPUs on the H100 partition. On CPUs, we ran it on 1 node with 1/2/3/4 processes on the General partition. Lastly, with the GFLOPs results, we will compute the SpeedUp using this formula

$$SpeedUp(P) = \frac{GFLOP_P}{GFLOP_1}$$

The reason why we used this version for our SpeedUp calculation instead of $SpeedUp(P) = \frac{T_1}{T_P}$ is because HPCG experiments always runs with a fixed amount of time specified by the user (RT). Hence, it is meaningless to use the timing to calculate the speed up

### 0.6 Gustafson scaling

A Weak Scaling study was conducted on both Easley's CPUs in the Debug Partition and L40S GPUs in the L40S partition. All tests were conducted on one node of their respective partitions. To keep a constant memory per process, we scaled our problem size $N^3$ up based on $N \propto \sqrt[3]{p}$ , s.t. $N = NX = NY = NZ$ for the GPU and CPU studies . The CPU study included 8 process sizes from 1 to 64 – p = {1,2,8,12,27,37,54,64} with respective sizes $N$ ={32,40,64,72,96, 104, 120, 128} – and started with a problem size of $N^3 = 32768$ (i.e. $N = 32$ for one process), and scaled up to a problem size of 2097152 with 64 processes, maintaining the problem's quantity for elements-per-process approximately equal to 32768. The GPU study included counts of 1 to 3 L40S GPUS; the GPU study started with a problem size of $N^3 = 884736$ (i.e. $N = 96$ for one gpu), up to 3 gpus – with respective sizes $N = \{96, 128, 144\}$ –with approximately 884736 elements-per-gpu. The serial, $f$, and and parallel fractions, $(1 - f)$, of Guffstafson's Law model for Speedup$(n) = (1 - f) + fn$ is derived from the aforementioned calculation in Amdahls's Law Methodology section. The parameter approximations are conducted by collecting the speedups for each study and fitting a line using the SciPy Python Library's `curve_fit(...)` function to each respective GPU and CPU study .

### 0.7 CPU vs GPU

We further evaluated the HPCG performance on both CPU and GPU performance across 6 problem sizes: $16^3$, $32^3$, $64^3$, $96^3$, $128^3$, $144^3$, each with 300 seconds of run time. GPU experiments were conducted on a single L40s node with one GPU while CPU experiments were executed on a single node with four processes total in the General partition. These setup configurations ensured the integrity when we evaluate how problem size influences computational performance on both models.
Additionally, we also perform experiments with different GPU count (2 and 3). This experiment will not only show how well GPU scales, but it will also help us understand whether communication or memory limits is the bottle neck.

### 0.8 Performance variation with node count

For the runs that focused on the effects of varying node counts on HPCG performance we ran HPCG using just CPUs. These runs were done using a fixed problem size of $(96, 96, 96)$ for a runtime of 120 seconds. With the specified parameters the runs were then conducted on Easley's General partition with the amount of nodes for each run ranging from 1-8.

### 0.9 Run Time Analysis

A Parameter Sweep was done on a single H100 for Easley to dedicate analysis on effects of runtime (sec.) , $RT$ =[150,300,400,500,600,800, 1200], with a constant problem size of $N = NX = NY = NZ = 128$. We sampled 15 throughput results (GFlops), via slurm array jobs.

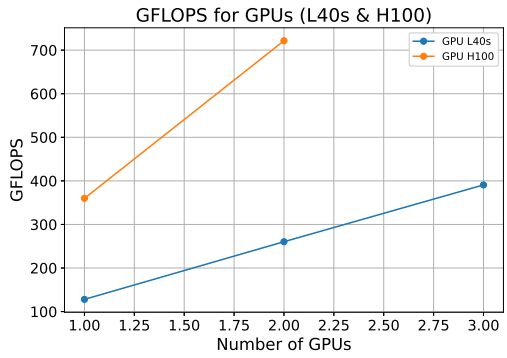
## Results

### 0.10 Amdahl scaling

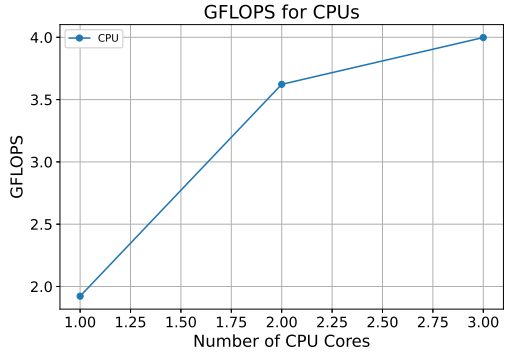
Our results are shown in table 1 and 2. We observe a significant increase in GFLOPs as more computational resources are allocated in to our runs on both CPU and GPU. On both GPUs partition, L40s and H100, our results demonstrate a near linear speedup when increasing the number of GPUs. For L40s, the speedup increases from 1 to $\approx 3$ times as the number of GPU grows. This results demonstrate a "perfect" linear performance, and excellent parallelization. Using the curve_fit with the least-squares method, we received an estimated serial fraction $\mathbf{f} \approx 2.87 \times 10^{-10}$. The smaller the serial fraction **f**, the larger the possible maximum speedup. Our **f** is extremely small which indicates that the serial portion of the workload is non-existent. This is caused by our sample size where we only have 3 data points to perform the analysis. We also experienced the same trend on our H100 partition, and expectedly, it performs significantly better than L40s. H100 is designed to be a high-end, more optimized partition for HPCG benchmark than L40s which explain the performance differences.
For CPU, although performance also increases as more processes are used, it does not scale as efficiently as the GPU. The speedup is slightly less, indicating a larger serial fraction ($\mathbf{f} = 0.1905 \approx 19\%$ ).

| *GPUs benchmark L40s partition* | | | | | | | |
|---|---|---|---|---|---|---|---|
| Node | GPUs | NX | NY | NZ | RT | GFLOPS | Speedup |
| 1 | 1 | 144 | 144 | 144 | 300 | 128.034 | 1 |
| 1 | 2 | 144 | 144 | 144 | 300 | 260.352 | 2.03 |
| 1 | 3 | 144 | 144 | 144 | 300 | 390.715 | 3.05 |
| *GPUs benchmark H100 partition* | | | | | | | |
| Node | GPUs | NX | NY | NZ | RT | GFLOPS | Speedup |
| 1 | 1 | 144 | 144 | 144 | 300 | 359.836 | 1 |
| 1 | 2 | 144 | 144 | 144 | 300 | 721.589 | 2.005 |
| *CPUs benchmark General partition* | | | | | | | |
| Node | P | NX | NY | NZ | RT | GFLOPS | Speedup |
| 1 | 1 | 144 | 144 | 144 | 300 | 1.92216 | 1 |
| 1 | 2 | 144 | 144 | 144 | 300 | 3.62256 | 1.885 |
| 1 | 3 | 144 | 144 | 144 | 300 | 3.99808 | 2.08 |

Table 1. Amdahl law results



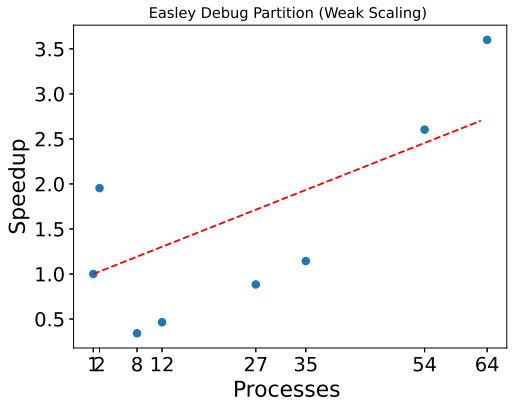

(a) Performance differences as GPU counts increase with fixed problem size for H100 and L40s



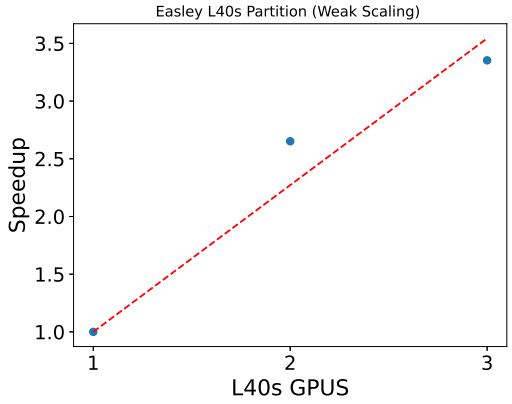

(b) Performance differences as process counts increase with fixed problem size for General

Fig. 1. Amdahl strong scaling

## 0.11 Gustafson Scaling Results




(a) Weak Scaling Study on CPU Processes in Debug Partition




(b) Weak Scaling Study on L40S GPUs in L40s Partition

Fig. 2. Weak Scaling Study on Easley's CPU vs GPU

**Fig 2**, demonstrates that curve fits; the curve fitting function was able to approximate the Gustafson Law of Speedup parameters as such : $\widehat{Speedup_{cpu}}(p) = 0.97255 + (0.02745)p$ ; $\widehat{Speedup_{cpu}}(p) = -0.27159 + (1.27159)p$. From these curve fitting models, we can approximate the serial portion for the CPU study as ~%2.75 and the parallel as ~%97.25; for the GPU study the serial portion is ~%1.27 and the parallel portion is ~%-0.27. **Figure (a)** contains a scatter of points with a suddenly shift around 2 and 8 processes likely due to the size of the problem being really small and due to the warnings given in the aforementioned ***.tmp** directory's files where the actual information for CPU studies was forced into - these referred to "suboptimal[ities]" given in the hpcg program we were running due to "Reference Versions" used for things like ComputeSPMV, ComputeDotProduct and other references in the program. The GPU study's parameters were skewed most likely due to the limited sample of gpu counts to test a curve fit - due to limit of resources per user for gpus on the Easley Cluster - hence, producing "impossible" parameters. Overall, the speedups are approximately linear as expected from Gustafson's Law.

## 0.12 CPU vs GPU

Besides the Amdahl and Gustafson experiments, we were also interested in conducting a third experiment where the allocated resources are fixed while the problem size increases for both CPU and GPU. The results are displayed in table 2, figure 3 and 4.

For the GPU runs, we again observed that configurations with more GPUs achieve higher performance overall. Additionally, the measured GFLOPs increased rapidly with problem size. The performance consistently peaked around $128^3$. However, the performance dropped for $144^3$. This trend persist across all 3 GPU count configurations.

At smaller sizes, CPU's performance is extremely poor, especially between $16^3$ and $64^3$. The performance only began to scale for larger problem sizes ($128^3$ and $144^3$). Interestingly, unlike the GPU runs, we did not experience the performance drop from $128^3$ to $144^3$. This is possibly because the larger problem size cases exceeded GPU memory capacity, where on the contrary, CPU has a larger system RAM which mean it will not experience any limitation. Another possible reason is related to the GPU's thread locality given the problem size. Thread locality refers to how well the data being accessed is arranged efficiently by the GPU's thread. When the data size does not "fit" efficiently in the GPU architecture, it can cause performance problem [1].

*GPUs benchmark L40s partition*

| Node | NX | NY | NZ | RT | GFLOPS(1) | GFLOP(2) | GFLOP(3) |
|---|---|---|---|---|---|---|---|
| 1 | 16 | 16 | 16 | 300 | 2.3691 | 4.11283 | 5.94385 |
| 1 | 32 | 32 | 32 | 300 | 20.3633 | 33.8253 | 50.2657 |
| 1 | 64 | 64 | 64 | 300 | 111.282 | 193.875 | 280.865 |
| 1 | 96 | 96 | 96 | 300 | 116.666 | 239.451 | 360.119 |
| 1 | 128 | 128 | 128 | 300 | 160.291 | 308.685 | 460.716 |
| 1 | 144 | 144 | 144 | 300 | 127.892 | 261.296 | 390.466 |

*CPUs benchmark General partition*

| Node | P | NX | NY | NZ | RT | GFLOPS |
|---|---|---|---|---|---|---|
| 1 | 4 | 16 | 16 | 16 | 300 | 0.0114382 |
| 1 | 4 | 32 | 32 | 32 | 300 | 0.0942355 |
| 1 | 4 | 64 | 64 | 64 | 300 | 0.382092 |
| 1 | 4 | 96 | 96 | 96 | 300 | 2.18137 |
| 1 | 4 | 128 | 128 | 128 | 300 | 4.18912 |
| 1 | 4 | 144 | 144 | 144 | 300 | 5.35195 |

Table 2. Performance results across various sizes and GPU count

## 0.13 Performance variation with node count

Another study we ended up conducting was the effect that increasing node count would have on the performance of HPCG. Particularly this was done using only CPUs as doing a similar study with GPUs would require some workarounds to get working and it would be limited to a max of three nodes at once which wouldn't be much data. looking at the data in figure 5 we did obtain, it can be seen that overall performance does increase with node count. However, it doesn't seem like the increase in performance between node counts is entirely constant. As when going from 3 to 4 nodes and from 5 to 6 nodes there's a notable lack of increase in performance, especially when compared to the massive increase in performance going from 4 to 5 nodes and from 6 to 7 nodes. This could be due to a wide variety of factors like communication bottlenecks, optimal resource usage, or simply just inconsistent data from a small sample size.

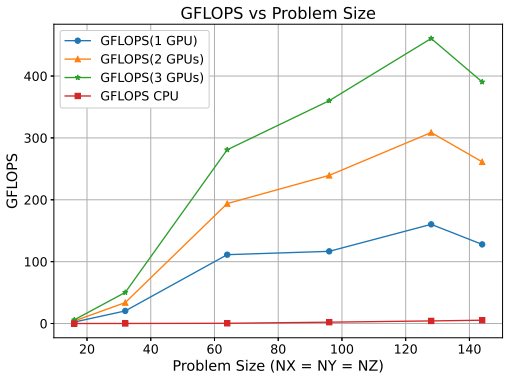



Fig. 3. CPU performance on HPCG benchmark with varying problem size.

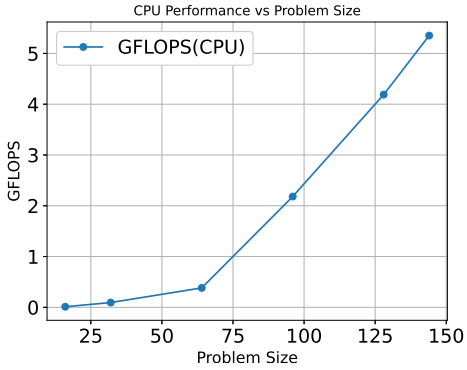



Fig. 4. CPU performance on HPCG benchmark with varying problem size.

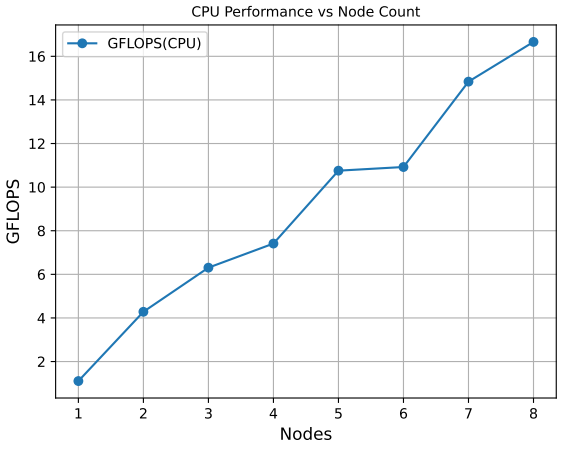



Fig. 5. CPU performance on HPCG benchmark with varying node counts.

## 0.14 Run time analysis

The boxplot runtimes in **fig 6.** have surprisingly remained consistent in spread, from ~405 to ~414.5 GFlops. This is possibly due to the problem size being rather small with reference to the single h100 we used (taking $128^3 \times 8 \approx 16.777216$ Mb in memory). The more noticeable difference is that the median for the 400 was rather low and had the smallest range, ~408.5 GFlops, indicating that about %50 of the throughput samples remained consistently above it. Increasing the sample points for each run to more than 100 would definitely increase the precision of our medians and variance of our throughput per runtime. In essence, it is difficult to assume that there is a significant **difference** in *smaller* problem sizes' throughput with varying RTs below and including $RT = 1200$ seconds.

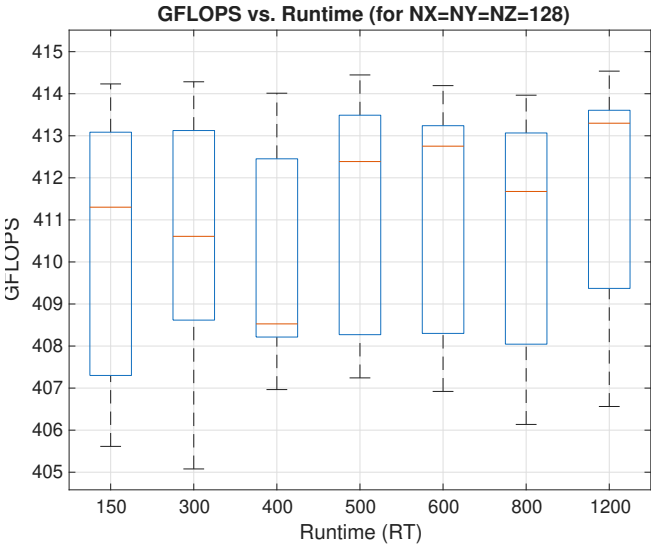



Fig. 6. Run Time Analysis On Easley : Single H100

## 0.15 Fastest HPCG Run

*Easley's GPUs H100 partion*

| Node | GPUS | NX | NY | NZ | RT | GFLOPS |
|---|---|---|---|---|---|---|
| 1 | 2 | 256 | 256 | 256 | 1500 | 1177.75 |

Table 3. Best performance on Easley, H100 partition

Additionally, we increased our SLURM runtime from 30 minutes to 1 hours. This allows us to perform more experiment with increasing RT values without. From our runtime analysis, it showed that by increasing our RT values, we experience some increase in performance. Hence, we ran multiple experiments, each with problem size $256^3$, with increasing RT values from 600 to 1800. We ended up with RT = 1500 as our best run.

## Conclusions

In retrospect, our results conducted over CPUs and GPUs' studies remained as expected - the GPU would outperform the CPU in most cases ; our studies of of 1-3 GPUs versus 1-64 processes resulted in GPUs outperforming with fewer in comparison to process counts, most likely due to the nature of GPUs' hardware to be better at matrix math. The Weak Scaling study demonstrated reasonable serial/parallel fraction approximations for the GPU given the lack of them at our availability, and demonstrated even more reasonable results for the CPUs, as data was abundant – resulting in an expected, small serial fraction for the benchmark. The Strong Scaling study of GPUs output reasonable approximations for the serial/parallel fractions of the benchmark given the limited resources for GPUs - indicating a similar trend as the Weak Scaling study, where the serial fraction is low. Moreover, The GPUs outperformed the CPUs given that our highest throughput was via H100s, and the CPUs struggled to gain more than 10+ GFlops; even our L40s competed with our H100s with the H100s having more throughput than the L40s on many occassions with the same dimensions and runtime parameters. This L40s v.s. H100 performance difference is most likely due to specialized architecture specific to H100s in reference to Conjugate Gradient for this benchmark allowing better performance. Unexpectedly, we saw a majo dip in all 1-3 L40s during various problem sizes (see **fig 3** & **table 2**), where it peaked at ~128 and started to decrease in throughput after - perhaps attributed to GPU vs CPU memory limits, or GPU thread locality.

## Author contributions

{**Loc Su**} worked on the initial HPCG set up and a working SLURM scripts for both GPU and CPU. Additionally, he worked on the Amdahl's scaling from writing the slurm script to running the experiment on Easley for both GPU and GPU. He also worked on the CPU vs GPU (including increasing GPU count) given various problem sizes. He and Eliud worked together on getting the fastest HPCG run. For the report, he wrote the General set up, Amdahl, and CPU vs GPU section in the Methodology. He also wrote the results and generated plots and tables for the Amdahl, CPU vs GPU, and the fastest HPCG run section.
{**Eliud García Reza**} worked on the Weak/Guffstafson Scaling and Run Time Analysis sections of this report, including conducting the parameter sweeping experiments on the L40S GPUS and Debug partition CPUs, writing the methodology and results sections, and producing their plots. He also wrote the conclusion.
{**Steven Upston**} worked on the CPU node and problem size studies, running the parameter sweeps and generating plots for both. Also wrote the abstract, introduction, and the CPU node studies results and methods sections .

## References

[1] Devashree Tripathy, Amirali Abdolrashidi, Laxmi Narayan Bhuyan, Liang Zhou, and Daniel Wong. 2021. PAVER: Locality Graph-Based Thread Block Scheduling for GPUs. *ACM Trans. Archit. Code Optim.* 18, 3, Article 32 (June 2021), 26 pages. doi:10.1145/3451164